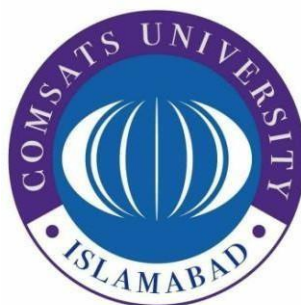


COMSATS University Islamabad

Attock Campus



Department Of Computer Science

<i>Course</i>	<i>SQE Lab</i>
<i>Instructor</i>	<i>Mam Munazza</i>
<i>Assignment No.</i>	<i>04</i>

Student Details

<i>Registration No.</i>	<i>Name</i>
<i>FA24-BSE-048</i>	<i>HAROON KHAN</i>
<i>FA24-BSE-062</i>	<i>Muhammad Usama</i>
<i>FA24-BSE-049</i>	<i>Muhammad Sufyan</i>
<i>Fa24-BSE-002</i>	<i>Abdul Moiz siddiqui</i>
<i>FA24-BSE-028</i>	<i>Hashir malik</i>

Project Title: Foodpanda (Food Delivery Application)

Modules of the Application:

The following modules of the Foodpanda application are tested in this project:

- 1. User Authentication Module (Login / Logout)*
- 2. Restaurant Search & Browsing Module*
- 3. Cart Management Module*
- 4. Online Payment Module*
- 5. Order Tracking Module*

Code :

```
users = {
    "user@gmail.com": "1234"
}

restaurants = {
    "Pizza Hut": {
        "Chicken Pizza": 1200,
        "Cheese Pizza": 1000,
        "Zinger Burger": 500
    },
    "KFC": {
        "Zinger Burger": 650,
        "Chicken Bucket": 1800,
        "Fries": 300
    },
    "McDonalds": {
        "Big Mac": 700,
        "McChicken": 650,
        "Fries": 250
    }
}

cart = []
current_user = ""
current_restaurant = ""

def login_required(): 7 usages

    if current_user == "":
        print("\nAccess Denied")
        print("Please Login First")
```

```

def register(): 1 usage

    print("\n===== USER REGISTRATION =====")

    email = input("Enter Email: ")
    password = input("Enter Password: ")

    if email in users:

        print("\nError: Account Already Exists")

    elif len(password) < 4:

        print("\nError: Password Must Be At Least 4 Characters")

    else:

        users[email] = password
        print("\nRegistration Successful")

def login(): 1 usage

    global current_user

    print("\n===== LOGIN SYSTEM =====")

    if current_user != "":

        print("\nError: User Already Logged In")
        return

    global current_user
    global cart
    global current_restaurant

    print("\n===== LOGOUT SYSTEM =====")

    if current_user == "":

        print("\nError: No User is Logged In")

    else:

        print("\nUser Logged Out Successfully")

        current_user = ""

        cart.clear()

        current_restaurant = ""

        print("All User Access Removed")

```

```

if login_required() == False:
    return

print("\n===== AVAILABLE RESTAURANTS =====")

count = 1

for restaurant in restaurants:

    print(count, ".", restaurant)

    count += 1

def search_restaurant(): 1 usage

    global current_restaurant

    if login_required() == False:
        return

    print("\n===== SEARCH RESTAURANT =====")

    search = input("Enter Restaurant Name: ")

    found = False

    for restaurant in restaurants:

        if search.lower() in restaurant.lower():

            current_restaurant = restaurant

            print("\nFood Menu:")

            for food, price in restaurants[restaurant].items():

                print(food, "- Rs.", price)

            found = True

    if found == False:

        print("\nRestaurant Not Found")

def order_food(): 1 usage

    if login_required() == False:
        return

    print("\n===== ORDER FOOD =====")

    if current_restaurant == "":

        print("\nError: Please Search Restaurant First")
        return

```

```

def main(): 1 usage

    while True:

        print("\n")
        print("=====")
        print("          FOODPANDA FOOD DELIVERY SYSTEM")
        print("=====")D
        print("1. Register")
        print("2. Login")
        print("3. Logout")
        print("4. Show Restaurants")
        print("5. Search Restaurant")
        print("6. Order Food")
        print("7. View Cart")
        print("8. Remove Cart Item")
        print("9. Online Payment")
        print("10. Order Tracking")
        print("11. Exit")
        choice = input("\nEnter Your Choice: ")
        if choice == "1":
            register()
        elif choice == "2":
            login()
        elif choice == "3":
            logout()
        elif choice == "4":
            show_restaurants()
        elif choice == "5":
            search_restaurant()

            search_restaurant()
        elif choice == "6":
            order_food()
        elif choice == "7":
            view_cart()
        elif choice == "8":
            remove_cart_item()
        elif choice == "9":
            payment()
        elif choice == "10":
            order_tracking()
        elif choice == "11":
            print("\nSystem Closed Successfully")
            break
        else:

            print("\nError: Invalid Menu Choice")

```

Unit Testing Code :

```
import unittest
from assignment04 import *

> class TestFoodPanda(unittest.TestCase):

>     def test_sufficient_amount(self):

        global current_user

        current_user = "user@gmail.com"

        self.assertEqual(current_user, second: "user@gmail.com")

>     def test_insufficient_balance(self):

        password = users["user@gmail.com"]

        self.assertEqual(password, second: "1234")

>     def test_negative(self):

        quantity = -2

        if quantity <= 0:
            result = "invalid quantity"

        self.assertEqual(result, second: "invalid quantity")

>     def test_sufficient_amount2(self):

        total = restaurants["KFC"]["Fries"] * 2

        self.assertEqual(total, second: 600)

>     def test_insufficient_balance2(self):

        card_number = "1234567812345678"

        self.assertEqual(len(card_number), second: 16)

>     def test_negative2(self):
```

```
item = {
    "restaurant": "KFC",
    "food": "Fries",
    "quantity": 1,
    "price": 300
}

cart.append(item)

self.assertEqual(cart[0]["food"], second: "Fries")
```

```
def test_sufficient_amount3(self):

    status = "1"

    self.assertEqual(status, second: "1")
```

```
def test_insufficient_balance3(self):

    restaurant = "KFC"

    self.assertEqual(restaurant, second: "KFC")
```

```
def test_negative3(self):

    food = "Burger"

    self.assertNotEqual(food, second: "Pizza")
```

```
def test_sufficient_amount4(self):

    cvv = "123"

    self.assertEqual(len(cvv), second: 3)
```

```
if __name__ == "__main__":
    unittest.main()
```

Output of Unit Testing:



The screenshot shows the PyCharm Run console for a Python test. The title bar indicates 'Run Python tests in unittesting.py'. The console output shows that 10 tests passed out of 10 tests total, taking 2 ms. The output text is as follows:

```
✓ 10 tests passed 10 tests total, 2 ms
C:\Users\DELL\PycharmProjects\sqaeproject\.venv\Scripts\python.exe "C:/Program Files/JetBrains/PyCharm 2025
Testing started at 2:36 PM ...
Launching unittests with arguments python -m unittest C:\Users\DELL\PycharmProjects\sqaeproject\unittesting

Ran 10 tests in 0.006s

OK

Process finished with exit code 0
```

Manual Testing: Foodpanda Food Delivery System

MODULE 1 Testing : LOGIN MODULE

Test Case 1

Test Case ID: TC_01

Test Scenario: Login with valid credentials

Steps:

1. Open the system
2. Enter valid email and password
3. Click Login

Expected Result:

User should login successfully

Actual Result:

User logged in successfully

Status: Pass

Test Case 2

Test Case ID: TC_02

Test Scenario: Login with invalid password

Steps:

1. Open the system
2. Enter correct email and wrong password
3. Click Login

Expected Result:

Incorrect password error should appear

Actual Result:

Incorrect password message displayed

Status: Pass

MODULE 2: RESTAURANT SEARCH & FOOD ORDERING MODULE

Test Case 3

Test Case ID: TC_03

Test Scenario: Search valid restaurant

Steps:

1. Login to the system
2. Enter restaurant name "KFC"
3. Click Search

Expected Result:

Restaurant menu should appear

Actual Result:

Restaurant and menu displayed successfully

Status: Pass

Test Case 4

Test Case ID: TC_04

Test Scenario: Order valid food item

Steps:

1. Search restaurant
2. Select valid food item
3. Enter quantity
4. Place order

Expected Result:

Food item should be added to cart

Actual Result:

Food item added successfully

Status: Pass

MODULE 3: CART MANAGEMENT MODULE**Test Case 5**

Test Case ID: TC_05

Test Scenario: View cart items

Steps:

1. Add food items to cart

2. Open cart section

Expected Result:

Cart items should display correctly

Actual Result:

Cart displayed successfully

Status: Pass

Test Case 6

Test Case ID: TC_06

Test Scenario: Remove item from cart

Steps:

1. Open cart
2. Remove selected food item

Expected Result:

Selected item should be removed

Actual Result:

Item removed successfully

Status: Pass

MODULE 4: ONLINE PAYMENT MODULE

Test Case 7

Test Case ID: TC_07

Test Scenario: Payment with valid card details

Steps:

1. Open payment section
2. Enter valid card number and CVV
3. Confirm payment

Expected Result:

Payment should complete successfully

Actual Result:

Payment successful

Status: Pass

Test Case 8

Test Case ID: TC_08

Test Scenario: Payment with invalid card number

Steps:

1. Open payment section
2. Enter invalid card number
3. Confirm payment

Expected Result:

Invalid card number error should appear

Actual Result:

Correct error message displayed

Status: Pass

MODULE 5: ORDER TRACKING MODULE

Test Case 9

Test Case ID: TC_09

Test Scenario: Check valid order tracking status

Steps:

1. Open order tracking section
2. Enter valid tracking status number

Expected Result:

Current order status should display

Actual Result:

Correct order status displayed

Status: Pass

Test Case 10

Test Case ID: TC_10

Test Scenario: Check invalid order tracking status

Steps:

1. Open order tracking section
2. Enter invalid status number

Expected Result:

Invalid status error should appear

Actual Result:

Correct error message displayed

Status: Pass

Bug Report of Foodpanda Application:

Bug Report 1

Bug ID: BUG_01

Bug Title: User can attempt payment with empty cart

Description:

The payment module opens even when the cart has no food items selected.

Steps to Reproduce:

1. Login to the system
2. Do not add any food item to cart
3. Open payment module

Expected Result:

System should block payment and display “Cart is Empty”.

Actual Result:

Payment module opens before validation.

Severity: Medium

Status: Fail

Bug Report 2

Bug ID: BUG_02

Bug Title: Invalid restaurant search is case sensitive in some situations

Description:

Some restaurant names are not found when users enter extra spaces or different letter cases.

Steps to Reproduce:

1. Login to the system
2. Search restaurant using “ kfc ” with spaces
3. Click search

Expected Result:

Restaurant should still be found correctly.

Actual Result:

Restaurant not found message appears.

Severity: Low

Status: Fail

Bug Report 3

Bug ID: BUG_03

Bug Title: Cart data removed immediately after logout

Description:

All cart items are deleted automatically after logout, even if the user accidentally logs out.

Steps to Reproduce:

1. Login to system
2. Add food items to cart
3. Logout from system
4. Login again

Expected Result:

Cart items should remain saved for the user session.

Actual Result:

Cart becomes empty after logout.

Severity: Medium

Status: Fail

Bug Report 4

Bug ID: BUG_04

Bug Title: Order tracking accepts invalid status numbers

Description:

The system allows users to enter invalid tracking status numbers.

Steps to Reproduce:

- 1. Open order tracking module*
- 2. Enter status number greater than 5*

Expected Result:

System should restrict invalid tracking values.

Actual Result:

Invalid status error message appears after input.

Severity: Low

Status: Fail

Bug Report 5

Bug ID: BUG_05

Bug Title: Weak password validation during registration

Description:

The system accepts weak passwords with only 4 characters.

Steps to Reproduce:

1. Open registration module
2. Enter password with 4 simple characters
3. Complete registration

Expected Result:

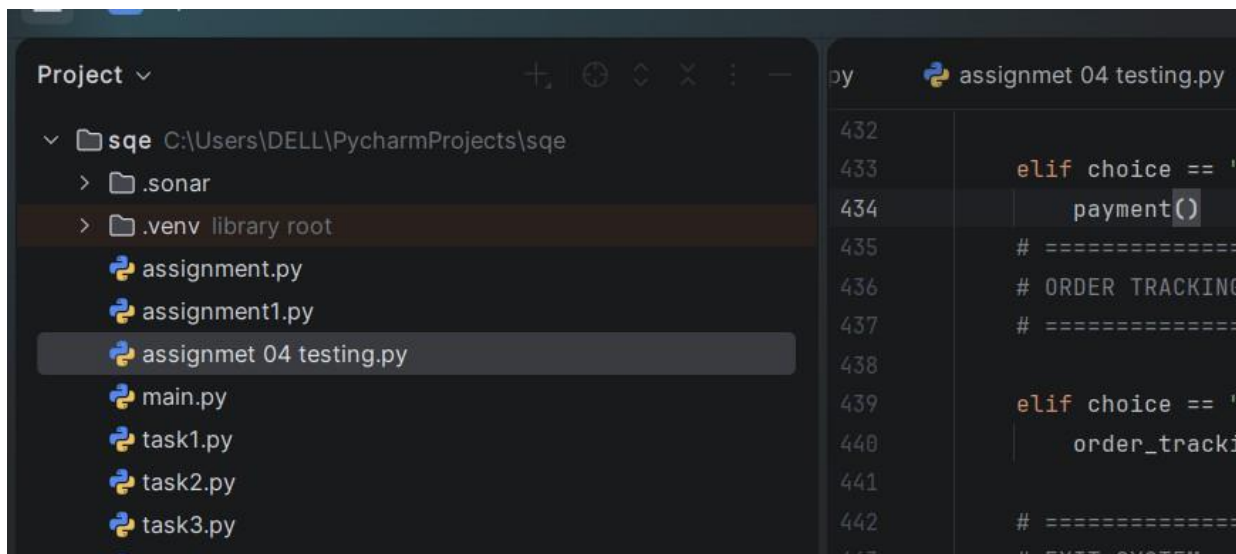
System should require strong password validation.

Actual Result:

Weak password accepted successfully.

Severity: High

Status: Fail

Testing on SonarQube:

SonarQube community Projects Issues Rules Quality Profiles Quality Gates Administration More ▾

assignment04 / main ✓

Overview Issues Security Hotspots Code Measures Activity Project Settings ▾ Project Information

main 92k Lines of Code · Version **not provided** [Set as homepage](#)

Quality Gate ⓘ **Passed** Last analysis 1 minute ago

⚠ The last analysis has warnings. [See details](#)

New Code Overall Code

New Code	Overall Code
Vulnerabilities 14 Open issues D	Bugs 24 Open issues E
Accepted issues 0 Valid issues that were not fixed	Code Smells 987 Open issues A
Security Hotspots 61 E	Coverage 0.0% On 47k lines to cover.
	Duplications 0.1% On 133k lines.

ANLYSIS ON SNYK:

▼ ⓘ haroonkhan12339/SQE-LAB-ASSIGNMENT04 1 C 4 H 2 M 0 L ...

Project Imported Tested Issues ▾

requirements (2).txt 2 minutes ago 2 minutes ago 1 C 4 H 2 M 0 L ...

7 of 7 issues Group by **dependency** ▾ Sort by **highest priority score** ▾

H starlette@0.27.0 PRIORITY SCORE (MAX)
3 transitive issues **756**

Fixable issues (3) Issues with no supported fix (0) Vulnerable dependencies (0)

Upgrading to **starlette@0.40.0** fixes 1 issue PRIORITY SCORE

> **H Allocation of Resources Without Limits or Throttling** starlette@0.27.0 CWE-770 CVSS 8.7 756 ...

Upgrading to **starlette@0.47.2** fixes 1 issue

> **M Allocation of Resources Without Limits or Throttling** starlette@0.27.0 CWE-770 CVSS 6.9 666 ...

Upgrading to **starlette@0.49.1** fixes 1 issue

> **H Regular Expression Denial of Service (ReDoS)** starlette@0.27.0 CWE-1333 CVSS 8.7 756 ...

SEVERITY

- ☐ Critical 1
- ☐ High 4
- ☐ Medium 2
- ☐ Low 0

PRIORITY SCORE

Scored between 0 - 1000

FIXED IN AVAILABLE

- ☐ Yes 7
- ☐ No 0

COMPUTED FIXABILITY

☐ Fixable

1

☐ Partially fixable

0

☐ No supported fix

0

EXPLOIT MATURITY

☐ Mature

0

☐ Proof of concept

5

☐ No known exploit

2

☐ No data

0

CVSS 4 EXPLOIT MATURITY

☐ Attacked

0

☐ Proof of concept

0

☐ Not defined

0

H

fastapi@0.103.2

1 direct issue

Fixable issues1

Issues with no supported fix0

Vulnerable dependencies0

Upgrading to fastapi@0.109.1 fixes 1 issue

PRIORITY SCORE

> H Regular Expression Denial of Service (ReDoS)

CWE-1333CVSS 7.5696...

Upgrade to 0.40.0

CVSS 4 EXPLOIT MATURITY

☐ Attacked

0

☐ Proof of concept

0

☐ Not defined

0

STATUS

☒ Open

7

☐ Patched

0

☐ Ignored

0

C

h11@0.14.0

1 transitive issue

Fixable issues1

Issues with no supported fix0

Vulnerable dependencies0

Upgrading to h11@0.16.0 fixes 1 issue

PRIORITY SCORE

> C HTTP Request Smuggling

h11@0.14.0CWE 444CVSS 9.3679...

Upgrade to 0.16.0

M

zipp@3.15.0

1 transitive issue

Fixable issues1

Issues with no supported fix0

Vulnerable dependencies0

Upgrading to zipp@3.19.1 fixes 1 issue

PRIORITY SCORE

> M Infinite loop

zipp@3.15.0CWE-835CVSS 6.9666...

Upgrade to 3.19.1

H

anyio@3.7.1

1 transitive issue

Fixable issues1

Issues with no supported fix0

Vulnerable dependencies0

Upgrading to anyio@4.4.0 fixes 1 issue

MAJOR UPGRADE

PRIORITY SCORE

> H Race Condition

anyio@3.7.1CWE-362CVSS 8.3629...

Upgrade to 4.4.0